

Formal Verification of the LZ77 and LZ78 Compression Algorithms

CS-428 project report

Alexander Shekhovtsov

Clément Tsedri

Ozair Faizan

2026-05-31

Abstract

Lempel-Ziv compression algorithms [LZ77; LZ78] are the backbone of many widely used compression algorithms, including DEFLATE, gzip, and zip. In this project, we modeled and formally proved the functional correctness of both algorithms in Rocq, alongside upper bounds on their worst-case compressed output lengths. Originally we proposed verification of low-level C implementation using the Verified Software Toolchain VST, however, verifying low-level memory operations and pointer arithmetic in VST is excessively time-consuming even for simple array manipulations, leading us to focus only on the high-level functional correctness.

Keywords: Formal verification, Lempel-Ziv, LZ77, LZ78, functional correctness, data compression.

Introduction

Data compression is widely-used in day-to-day and security-sensitive applications, yet its correctness often relies on manual testing rather than formal verification.

LZ77 Algorithm

The LZ77 algorithm operates over a sliding window of previously seen bytes. During compression, the algorithm scans the input and, at each position, searches the window for the longest match to the upcoming bytes. If a match of length at least 3 is found, it emits a reference token (length, offset), where offset is the distance to the start of the match in the window. Otherwise, it emits a literal token containing the current byte. These references are packed into two bytes: 4 bits for the length and 12 bits for the offset. Therefore, they are limited to the range [3, 18] and [3, 4098] respectively, as a bias of +3 is used.

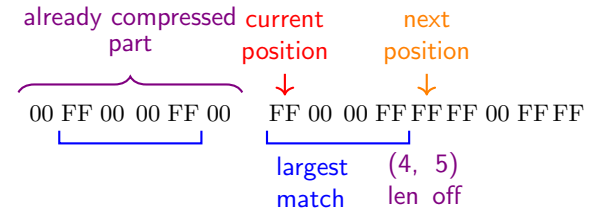


Figure 1: Illustration of LZ77 compression algorithm. The blue segment is the largest match in the window. It is of length 4 and with an offset of 5 from the current position.

LZ78 Algorithm

LZ78 takes a dictionary-based approach. It maintains an explicit dictionary of previously seen bit sequences, starting with a single empty entry. At each step, the algorithm finds the longest prefix of the remaining input that already appears in the dictionary, then emits a token (index, next_bit) pointing to that dictionary entry and the following bit. The concatenation of the matched prefix and the new bit is then added to the dictionary.

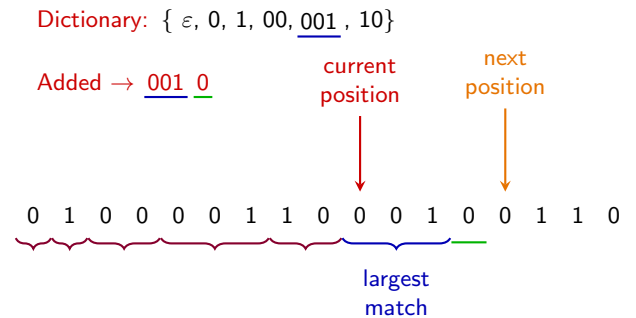


Figure 2: Illustration of LZ78 compression algorithm. The blue segment is the largest match with a word from dictionary. The string 0010 is referred to as a new *phrase*. Note that as a result of its work, the algorithm partitions the input string into distinct phrases (purple segments).

Problem Statement

The goal of this project is to verify end-to-end correctness of the compression algorithm by proving that decompression recovers the original data from the compressed data. Additionally, provide an upper bound on the compressed output size.

Approach

Compression proceeds in two passes, in the first pass, the input byte/bit sequence is converted to a list of abstract tokens and in the second pass, the token list is serialized to bytes/bits. Decompression reverses these steps. This two-level design cleanly separates the compression logic from the byte/bit encoding and simplifies the reasoning.

LZ77 Implementation

Token Definition

The first pass of LZ77 produces a list of Tokens. A token is either a literal byte or a back-reference.

```
Inductive Token :=  
  | Lit (b: byte)  
  | Ref (length offset: nat).
```

A token is considered *valid* if a reference token satisfies the constraints on length and offset. A key lemma is that compression produces valid tokens.

Matching

The matching module implements the sliding-window search. `find_largest_match` find the largest match that satisfies the constraints on length and offset. The key correctness lemma about matching is `find_largest_match_eq` which states that the slice of before at the reported offset byte-wise equals the corresponding prefix of after of the reported length.

Compression and Decompression

The compression is defined by the auxiliary function `compress_aux before after l`, where `before` is the bytes already processed, `after` is the remaining input, and `l` is a skip counter that advances past the bytes already covered by the most recent back-reference. When the skip counter is zero, it either emits a literal or a back-reference and sets the skip counter to length-1 to skip over the matched bytes. The decompression `decompress_aux acc token` mirrors the structure, a literal token appends its byte to `acc`, a reference token

copies length bytes from `acc[k; k + length)` where $k = |\text{acc}| - \text{offset}$.

Token Serialization

The second pass serializes tokens to bytes. Each valid reference token is packed into two bytes by `token_to_bytes`. The upper nibble of the first byte holds length-3, the lower nibble holds the high byte of offset-3, and the second byte holds its low byte. A flag bit is required to distinguish between bytes encoding literals and bytes encoding references, therefore, `tokens_to_bytes_chunk` prepends a flag byte for the following 8 tokens encoding. Finally, `tokens_to_bytes` recursively calls `tokens_to_bytes_chunk` to serialize the token list. Moreover, the inverses of these functions are also defined for decompression.

As a verification of a C implementation was expected, we needed to encode the length of the original input, so that a large enough buffer could be allocated. Therefore, a *variable-length encoding* is prepended to the compressed bytes, however it is never actually used by the decompression algorithm (since in Rocq we have access to the length of a list).

LZ78 Implementation

Token Definition

LZ78 operates over bits rather than bytes to simplify the upper bound theorem. Again, the first pass produces a list of Tokens.

```
Inductive Token :=  
  | Tok (index: nat) (phr : list bool) (next: bool)  
  | Last (index: nat) (phr : list bool).
```

`Tok` carries the dictionary index of the matched prefix `phr`, and the following bit `next`. Last token may be used as the very last token if the match exhausts the remaining input buffer. The phrases `phr` are for proof purposes only (when converting tokens to bits, `phr` is not used).

Dictionary and Matching

The dictionary has type `list (list bool)`, initialised as `empty_dict := [[]]`. The function `find_largest_prefix` scans the dictionary for the entry with the largest length that is a prefix of the remaining input, returning its index and length.

Compression and Decompression

Both the compression and decompression algorithms are similar enough to LZ77 to not require further detail.

Token Serialization

The second pass serializes tokens to bits. Each token's index is encoded with `num_bits_for_dict` = $\lceil \log_2 |\text{dict}| \rceil$ bits. The deserializer computes $|\text{dict}|$ by counting the number of tokens, as each token equates to a dictionary entry. Similar to LZ77, `tokens_to_bits` and its left-inverse `bits_to_tokens` are defined.

Results

LZ77 Correctness

The correctness theorem is proved in two steps. In the first step

Theorem `compress_decompress_correctness_aux`:

```
forall n bef aft, length aft ≤ n ->
  decompress_aux bef (compress_aux bef aft 0)
    = bef ++ aft.
```

is proved by strong induction on n (the length of `aft`). The [proof](#) proceeds by case analysis on whether a match is found. In the literal case the goal follows immediately. In the reference case, several lemmas about the `slice` function and the correctness lemmas about `find_largest_match` are used. `compress_rolls` which shows that advancing the skip counter is equivalent to prepending the skipped bytes to before and compressing the remainder, is a key lemma.

The second step is to prove the full round-trip:

Theorem `compress_to_bytes_correctness`:

```
forall s,
  decompress_from_bytes
    (compress_to_bytes s) = s.
```

Since the compression logic is separated from the serialization logic, we [proved](#) that

Theorem `tokens_to_bytes_to_tokens_correctness`:

```
forall t, forall valid_token t ->
  bytes_to_tokens (tokens_to_bytes t) = t.
```

As `compress_to_bytes` and `decompress_from_bytes` are defined as `tokens_to_bytes` $\circ$ `compress` and `decompress` $\circ$ `bytes_to_tokens` (ignoring the length encoding), the proof of the above theorem follows immediately from the two correctness lemmas.

LZ77 Upper Bound

The worst case of LZ77 occurs when no match is ever found during the compression process and every byte is emitted as a literal. Since the encoding of each literal requires an additional flag bit, the worst case upper bound is (ignoring length encoding) given by

$$|\text{compress_to_bytes } s| \leq \left\lceil \frac{9|s|}{8} \right\rceil.$$

To prove this bound we first assign each token a weight: 1 for literals and 3 for references. The weight represents the smallest prefix each token can consume. The `tokens_to_bytes_bounded_by_weight` and `compress_weight_upper_bound` are used to prove the chain of inequalities

$$|\text{compress_to_bytes } s| \leq \left\lceil \frac{9 \sum_i \text{weight } t_i}{8} \right\rceil \leq \left\lceil \frac{9|s|}{8} \right\rceil.$$

LZ78 Correctness

The correctness proofs are similar enough to LZ77 to not require further detail.

LZ78 Upper Bound

We [proved](#) an asymptotic upper bound on the compressed size: $|\text{compress_to_bits } s| \leq |s| + o(|s|) = |s| + 21|s|/\log_2 |s|$.

Theorem `compress_to_bits_bound`: **forall** s n k ,

```
n = length (compress s) ->
k = length s ->
10 ≤ Nat.log2 n ->
12 ≤ Nat.log2 k ->
length (compress_to_bits s)
  ≤ k + 21 * k / Nat.log2 k.
```

The proof uses two inequalities. The [first](#) bounds the bit length in terms of the number of tokens, since each token at dictionary size d requires $\lceil \log_2 d \rceil$ bits. The [second](#) is a lower bound on $|s| \geq m \cdot (\log_2 m - 2)$ in terms of the number of tokens m . The theorem is proved using the fact that $s = p_1 + p_2 + \dots + p_m$ is [partitioned into distinct phrases](#). We establish the following sequence of inequalities that allows us to lower bound $|s|$.

$$\begin{aligned} |s| &= \sum_{i=1}^m |p_i| \stackrel{(1)}{\geq} \sum_{i=1}^m \sum_{k=1}^{\log_2 m} I(|p_i| \geq k) \stackrel{(2)}{=} \sum_{k=1}^{\log_2 m} \sum_{i=1}^m I(|p_i| \geq k) \\ &\stackrel{(3)}{\geq} \sum_{1 \leq k \leq \log_2 m} (m + 1 - 2^k) \stackrel{(4)}{\geq} m \cdot (\log_2 m - 2) \end{aligned}$$

The bound (1) (`sum_over_list_sum`) is established using lemma `sum_over_indicator` that allows to express a number as the sum of indicators. We prove that two summations can be swapped in `sum_row_sum_col` (2). In `distinct_length_ge_n_lower_bound`, we prove that at least $m - 2^{k+1} + 1$ strings from $p_1, \dots, p_m$ have length at least k , allowing us to conclude (3). Finally, `sum_over_log2_lower_bound` proves (4). Using the resulting `lower bound` and some `asymptotic bounds`, the `upper bound` is established.

Limitations

Our original proposal included verification of a C implementation using VST. We were unable to complete this, for several reasons mainly related to VST itself. VST is known to be labor-intensive where simple array manipulations may require substantial efforts to prove. This is well-documented by Andrew Appel [App15, p. 16] who participated in the creation of the framework. Furthermore, our two pass approach simplified the formalization, however C implementations perform a single pass writing directly into the output byte buffer. We also attempted to write recursive C functions to match the Rocq fixpoints, but this along with other attempts at `VST proofs` were not too successful.

Timeline

The proofs of LZ77 correctness and upper bound were done a week earlier than planned. Then, we had 3 weeks (originally 2) to prove the correctness of our C implementation, however, after dedicating two weeks to VST without any progress, we made the decision to pivot our scope. We reallocated our remaining schedule to formally model, prove the correctness, and an asymptotic upper bound of the LZ78 algorithm instead.

At the start of each major step (new theorem/new algorithm) we laid out the functions to implement and lemmas to prove. The proofs were started while pair coding, and then finalized separately. LZ77 and LZ78 upper bounds were done by Alexander Shekhovtsov. LZ78 simple upper bound was done by Clément Tsedri. LZ77 and LZ78 serialization, LZ78 correctness and the asymptotic bound section were done by Ozair Faizan. All other lemmas and theorems had roughly equal contribution.

Related work

Our project closely aligns with existing data compression verification efforts, most notably `lean-zip` which is an implementation and verification of the LZ77 algorithm

within the DEFLATE standard written in Lean 4. Our work differs by targeting both LZ77 and LZ78 in Rocq, and providing concrete upper bounds on compressed size.

There is a `catalog of examples` of VST-based verifications, most notably, [App15]. These projects confirm the significant cost of VST-based verification, consistent with our experience.

Future work: Asymptotic Optimality

Lempel and Ziv [LZ78] proved that LZ78 algorithm is at least as efficient as any finite state (FS) compressor. More precisely, for any infinite string $s \in \{0, 1\}^{\mathbb{N}}$ and any FS compressor C with q states, the upper limit $\limsup_n |LZ78(s[1 : n])|/n$ is bounded above by the lower limit $\liminf_n |C(s[1 : n])|/n$.

To prove the theorem authors use the fact that LZ78 compressor partitions the initial string into distinct phrases $p_1 + p_2 + \dots + p_m = s$. They show that a finite state compressor C should spend at least $m \cdot (\log_2 m - O(1))$ bits to compress s and that the length of the initial string is at least $m \cdot (\log_2 m - 2)$, which we have proved in this work. Therefore, to complete the proof of the asymptotic optimality theorem, it remains to only prove the former result. This allows to show that the length of the compressed string is not only not larger than the length of the initial string, but also can be significantly smaller if there is a FS compressor that can greatly reduce its length.

Conclusion

We have presented proofs for the correctness of our implementation of LZ77 and LZ78 compression algorithms. We additionally proved upper bounds on compressed sizes.

The formalization demonstrates that a two-pass architecture (compression to tokens, then token serialization) is well-suited for formal verification as it allows independent proofs at each level.

Our experience with VST suggests that verifying a low-level C implementation of LZ77 is feasible but requires substantially more effort than the functional formalization.

This work provides a solid, foundation for Lempel-Ziv compression that can serve as a basis for further verification of compression-related software, including C implementations, streaming variants, and higher-level algorithms built on the LZ family.

AI use disclosure

- Alexander Shekhovtsov
 - Used AI assistants to understand technical details of Verified Software Toolchain.
 - Used AI to help prove simple technical lemmas.
 - Used AI to generate tikz latex diagrams.
- Ozair Faizan
 - The LLMs are capable of proving some simple lemmas, however, for more challenging proofs they fails.
 - Nevertheless, they were quite helpful for pen and paper proofs and brainstorming ideas.
- Clément Tsedri
 - ChatGPT is useful for small proofs or some formalisation details, however they are not yet able to output correct proofs for more challenging theorems outright

References

- [App15] Andrew Appel. “Verification of a Cryptographic Primitive: SHA-256”. In: *ACM Transactions on Programming Languages and Systems* 37 (Apr. 2015), pp. 1–31. DOI: [10.1145/2701415](https://doi.org/10.1145/2701415).
- [LZ77] Abraham Lempel and Jacob Ziv. “A Universal Algorithm for Sequential Data Compression”. In: *IEEE Transactions on Information Theory* 23.3 (1977), pp. 337–343. DOI: [10.1109/TIT.1977.1055714](https://doi.org/10.1109/TIT.1977.1055714).
- [LZ78] Abraham Lempel and Jacob Ziv. “Compression of individual sequences via variable-rate coding”. In: *IEEE Transactions on Information Theory* 24.5 (1978), pp. 530–536. DOI: [10.1109/TIT.1978.1055934](https://doi.org/10.1109/TIT.1978.1055934).